PRÁCTICA

Práctica del RA1: Operación DataShield

David Rodera - Marco Gallego

Administración de Sistemas Gestores de Bases de Datos (ASGBD) - 22/10/2026
2º Administración de Sistemas Informáticos en Red (2ºASIR)

Índice

0. Introducción **3**

0.1. Contexto 3
0.2. Objetivos 3

1. Diseño e Implementación de la Base de Datos **4**

1.1. Estructura de la Base de Datos 4
1.2. Creación de la Base de Datos 5
1.3. Inserción de datos 9

2. Creación de las Vistas Seguras **12**

2.1. Creación vista_incidencias_analista 12
2.2. Creación vista_sistemas_sin_ip 13
2.3. Creación vista_incidencias_soporte 14

3. Creación de Usuarios y Modelo RBAC **15**

3.1. Creación de Roles 15
3.2. Creación de Usuarios 16

4. Verificación de la configuración asignada **18**

4.1. Pruebas funcionales básicas 19
4.2. Pruebas de separación de datos 21

0. Introducción

0.1. Contexto

El departamento de seguridad del instituto ha detectado una brecha: un antiguo alumno filtró datos por tener acceso total a una base de datos compartida. Tu equipo, formado por dos administradores junior, debe restablecer la seguridad creando un sistema de usuarios, roles y vistas seguras.

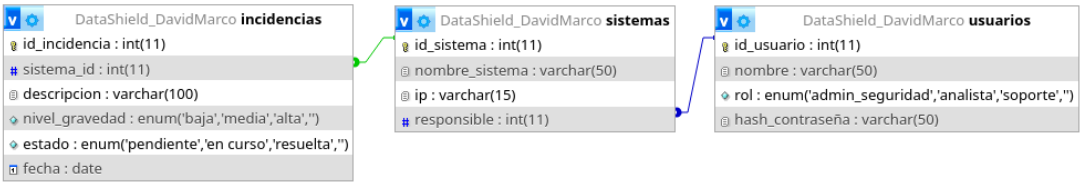
Este reto se llama **Operación DataShield** porque se refiere precisamente a construir un "escudo de protección" combinando dos pilares de seguridad en bases de datos:

1. Abstracción mediante **Vistas Seguras**: Interponer una capa intermedia para que los usuarios nunca consulten las tablas físicas reales (ocultando IPs internas yhashes de contraseñas).
2. **Modelo RBAC** (Role-Based Access Control (Control de Acceso Basado en Roles)) para roles: Restringir el acceso a nivel de usuario asignando únicamente los roles estrictamente necesarios (Principio de Mínimo Privilegio).

0.2. Objetivos

1. Creación de la base de datos **DataShield_DavidMarco**.
2. Configurar un sistema de roles y vistas seguras que permita diferentes niveles de acceso:

- **Analistas**: solo lectura de datos no sensibles.
- **Técnicos de soporte**: modificación de registros de incidencias.
- **Administradores**: gestión completa y auditoría.



1. Diseño e Implementación de la Base de Datos

1.1. Estructura de la Base de Datos

Nuestra base de datos **DataShield_DavidMarco** estará compuesta por tres tablas: **usuarios**, **sistemas** e **incidencias**.

Tabla usuarios

Campo	Tipo	Descripción
id_usuario	INT	Idenficador único
nombre	VARCHAR(50)	Nombre y apellidos del usuario
rol	ENUM	Rol que desempeña (admin_seguridad, analista, soporte)
hash_contraseña	VARCHAR(50)	Contraseña cifrada o hash (no en claro)

Para la columna rol usaremos **ENUM** en vez de **VARCHAR**, ya que **ENUM()** es un objeto de cadena que limita los valores permitidos de una columna a una lista fija y predefinida, y como solo 3 valores a insertar en esa columna sería más correcto.

Tabla sistemas

Campo	Tipo	Descripción
id_sistema	INT	Idenficador único
nombre_sistema	VARCHAR(50)	Nombre del servidor o aplicación
ip	VARCHAR(15)	Dirección IP del sistema
responsible	INT	Usuario responsable del sistema (FK a usuarios)

En la columna ip, tiene un tipo **VARCHAR(15)** ya que una dirección IP está compuesta por 4 octetos de números del 0 al 255 separados por puntos, por lo que el máximo de caracteres es de 15.

Como hay IPs que no ocupan justo 15 caracteres el tipo **VARCHAR** que permite un número de caracteres menor o igual a lo indicado, si todas las IPs ocupasen 15 caracteres la mejor opción sería **CHAR**.

Tabla sistemas

Campo	Tipo	Descripción
id_incidencia	INT	Idenficador único
sistema_id	INT	Sistema afectado (FK a sistemas)
descripcion	VARCHAR(100)	Descripción del problema
nivel_gravedad	ENUM	Puede ser “baja”, “media” o “alta”
estado	ENUM	Puede ser “pendiente”, “en curso”, “resuelta”
fecha	DATE	Fecha de detección

Usaremos el tipo **ENUM** en las columnas **nivel_gravedad** y **estado** ya que tienen pocos valores a insertar.

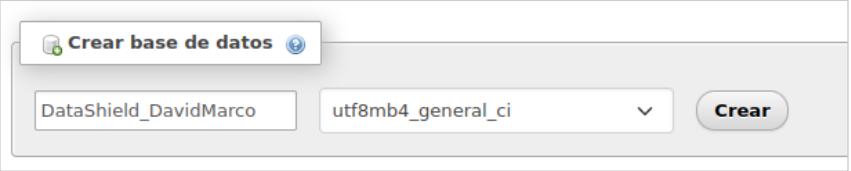
1.2. Creación de la Base de Datos

Creación inicial

Para empezar con la creación de la base de datos, se utiliza la instrucción `DROP DATABASE IF EXISTS base_datos;` con el fin de eliminar cualquier versión previa que pueda existir con ese mismo nombre en el servidor. Esto asegura que la base de datos se cree completamente desde cero, evitando posibles errores de conflicto por tablas o datos antiguos. Posteriormente crearemos la base de datos, e indicaremos que queremos usarla.

```sql
DROP DATABASE IF EXISTS DataShield_DavidMarco;
CREATE DATABASE DataShield_DavidMarco;
USE DataShield_DavidMarco;
```

También podemos crearla de manera gráfica.



Creación tabla usuarios

Podemos crear la tabla desde comandos:

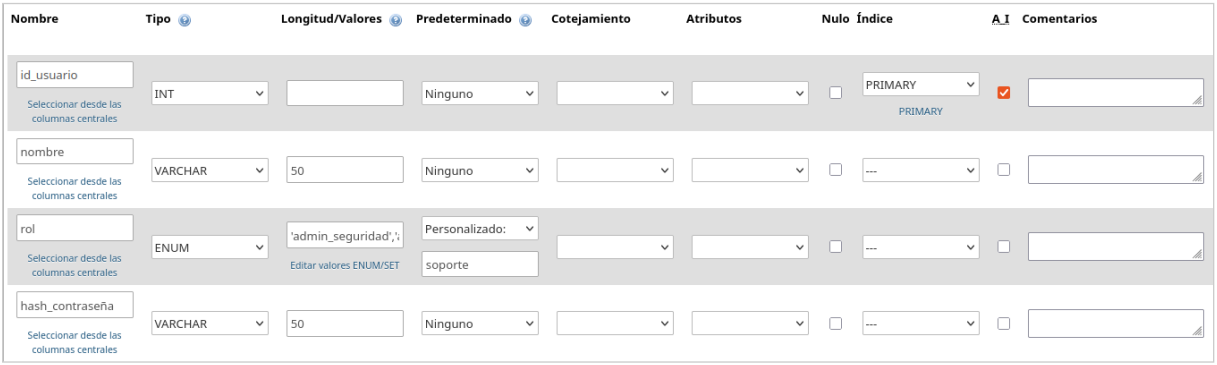
```sql
CREATE TABLE usuarios (
    id_usuario INT PRIMARY KEY AUTO_INCREMENT,
    nombre VARCHAR(50) NOT NULL,
    rol ENUM('admin_seguridad', 'analista', 'soporte') DEFAULT 'soporte',
    hash_contraseña VARCHAR(50) NOT NULL
);
```

Se usará **AUTO_INCREMENT** para la clave primaria, esto hará que se genere automáticamente un número único cada vez que se inserte un nuevo registro en una tabla.

Usaremos la restricción **NOT NULL**, de esta forma los valores insertados no pueden ser nulos.

Usaremos la cláusula **DEFAULT** para que si se inserta un valor nulo en rol, se autoasigne el rol soporte.

O también de forma gráfica:



Marcando la casilla **A_I** asignaremos el **AUTO_INCREMENT**, y con la opción **Personalizado** en **Predeterminado** marcaremos el **DEFAULT**.

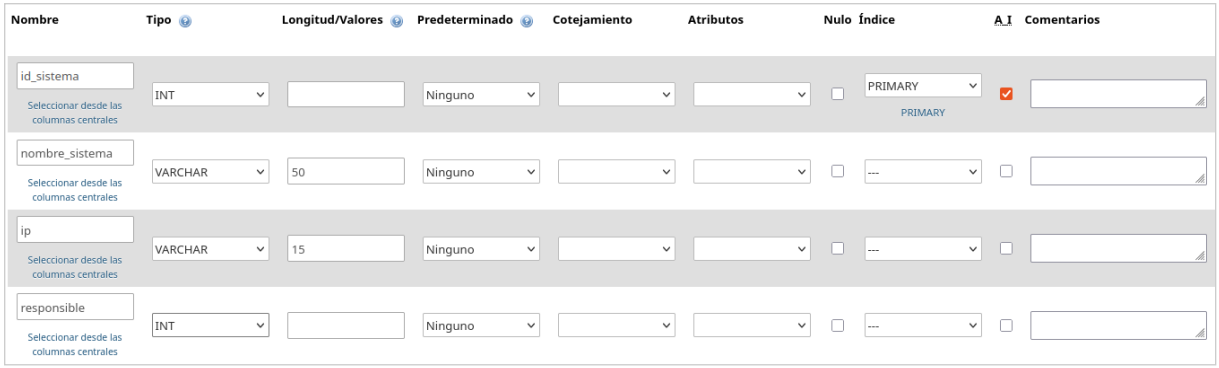
Creación tabla sistemas

Podemos crear la tabla desde comandos:

```sql
CREATE TABLE sistemas (
    id_sistema INT PRIMARY KEY AUTO_INCREMENT,
    nombre_sistema VARCHAR(50) NOT NULL,
    ip VARCHAR(15) NOT NULL,
    responsible INT,
    FOREIGN KEY (responsible) REFERENCES usuarios (id_usuario)
    ON DELETE RESTRICT
    ON UPDATE RESTRICT
);
```

> Referenciaremos la columna **responsible** como **FK** a **usuarios**.

O también de forma gráfica:



Para referenciar la FK de forma gráfica debemos acceder a la tabla **sistemas** –> **Estructura** –> **Vista de relaciones** y añadiremos la referencia.



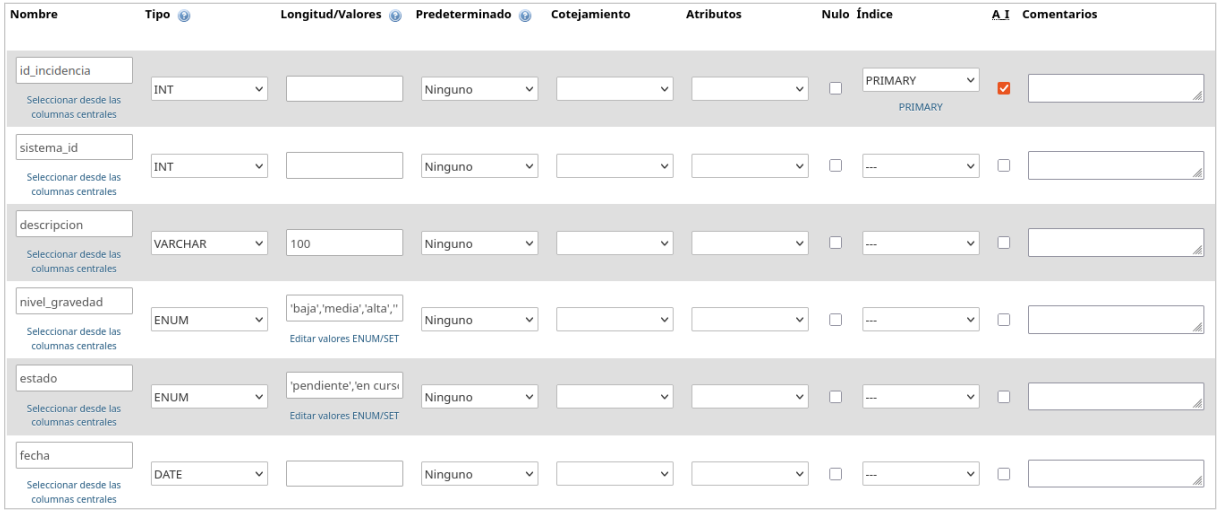
Creación tabla incidencias

Podemos crear la tabla desde comandos:

```sql
CREATE TABLE incidencias (
    id_incidencia INT PRIMARY KEY AUTO_INCREMENT,
    sistema_id INT,
    descripcion VARCHAR(100),
    nivel_gravedad ENUM('baja', 'media', 'alta'),
    estado ENUM('pendiente', 'en curso', 'resuelta'),
    fecha DATE NOT NULL,
    FOREIGN KEY (sistema_id) REFERENCES sistemas(id_sistema)
    ON DELETE RESTRICT
    ON UPDATE RESTRICT
);
```

> Referenciaremos la columna **sistema_id** como **FK** a **sistemas**:

O también de forma gráfica:



Para referenciar la FK de forma gráfica debemos acceder a la tabla **incidencias** –> **Estructura** –> **Vista de relaciones** y añadiremos la referencia.



1.3. Inserción de datos

Tabla usuarios

Podemos insertar los datos desde comandos:

```sql
INSERT INTO usuarios (nombre, rol, hash_contraseña) VALUES
('Laura Gómez', 'admin_seguridad', 'a3f4c9'),
('Carlos Pérez', 'analista', 'bc97d1'),
('Marta Ruiz', 'soporte', 'ef45aa');
```

> Como hemos puesto la clave **AUTO_INCREMENT**, no hay necesidad de introducir valores en la columna **id_usuario**.

O también de forma gráfica:

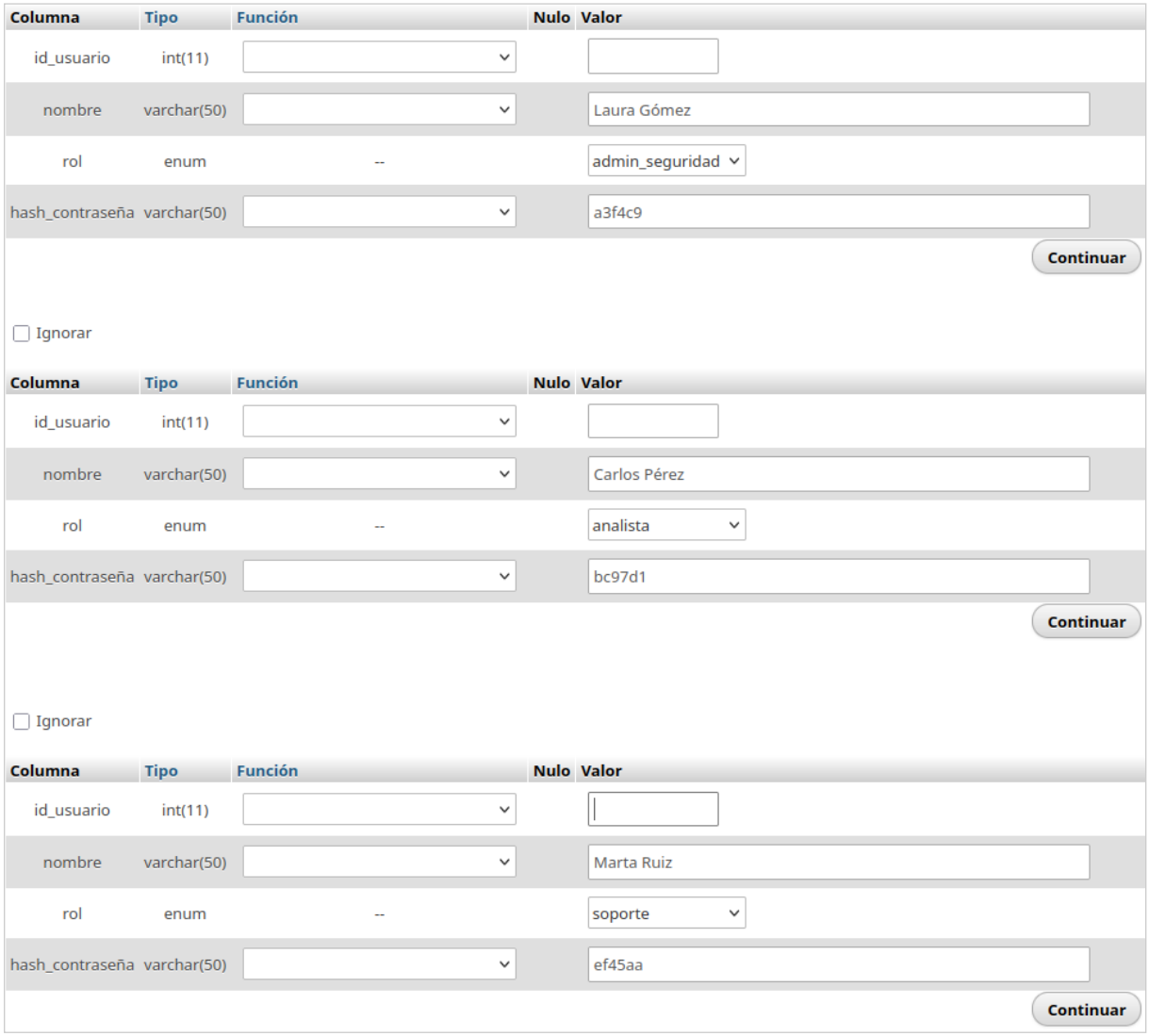


Tabla sistemas

Podemos insertar los datos desde comandos:

```sql
INSERT INTO sistemas (nombre_sistema, ip, responsible) VALUES
('ServidorCorreo', '192.168.10.5', 3),
('WebInstituto', '10.0.0.22', 2);
```

> Como hemos puesto la clave **AUTO_INCREMENT**, no hay necesidad de introducir valores en la columna **id_sistema**.

O también de forma gráfica:

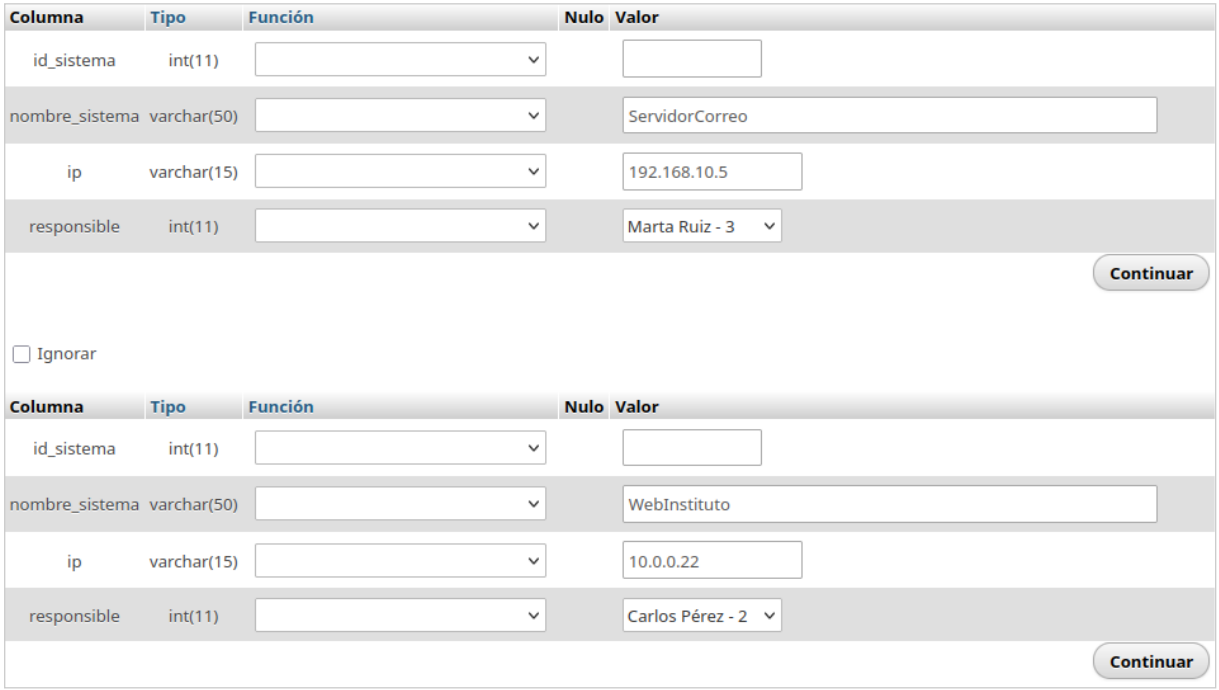


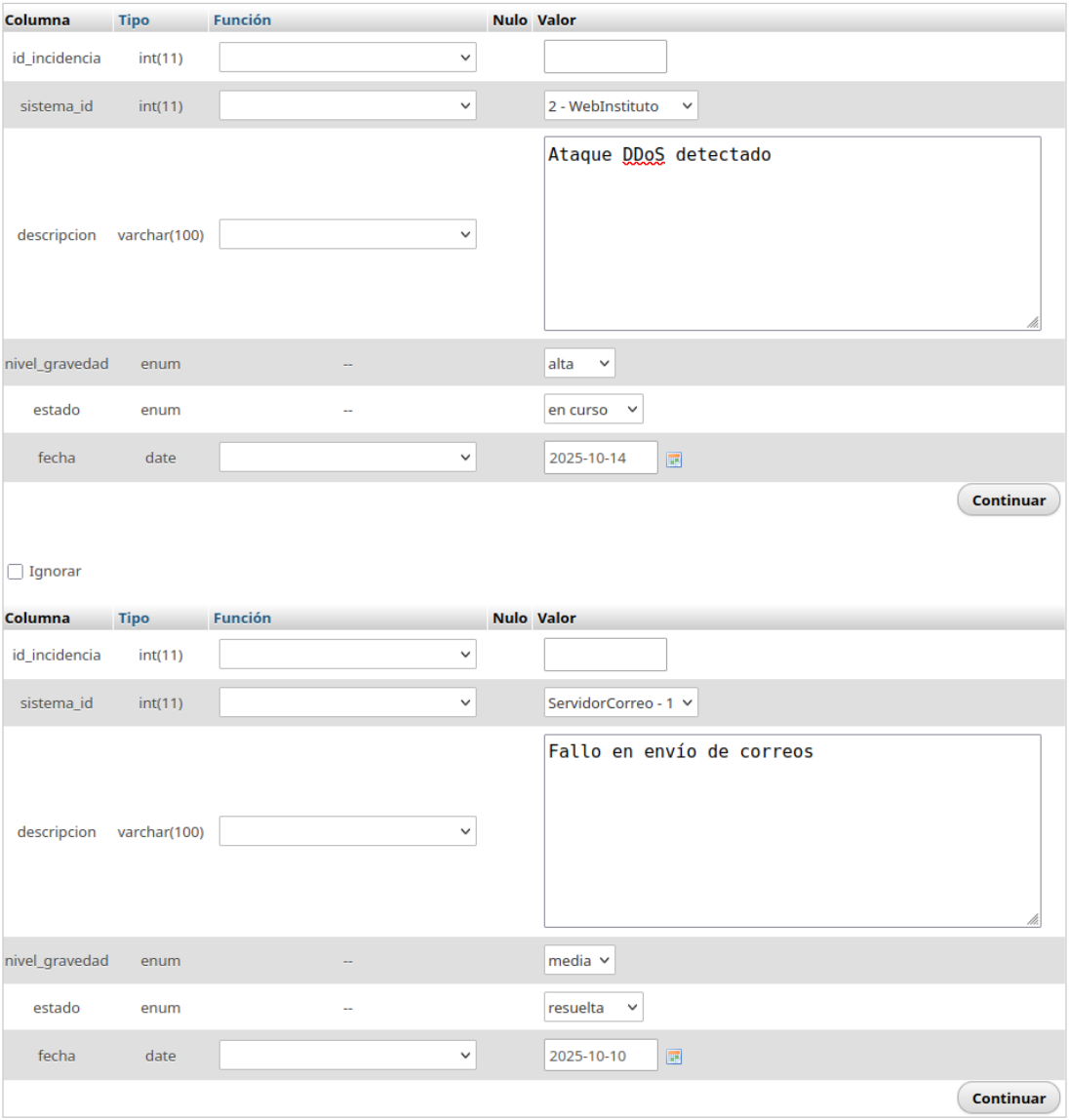
Tabla incidencias

Podemos insertar los datos desde comandos:

```sql
INSERT INTO incidencias (sistema_id, descripcion, nivel_gravedad, estado, fecha)
VALUES
(2, 'Ataque DDoS detectado', 'alta', 'en curso', '2025-10-14'),
(1, 'Fallo en envío de correos', 'media', 'resuelta', '2025-10-10');
```

> Como hemos puesto la clave **AUTO_INCREMENT**, no hay necesidad de introducir valores en la columna **id_incidencia**.

O también de forma gráfica:



2. Creación de las Vistas Seguras

Las vistas actuarán como un escudo intermedio para que los usuarios no accedan directamente a las tablas físicas.

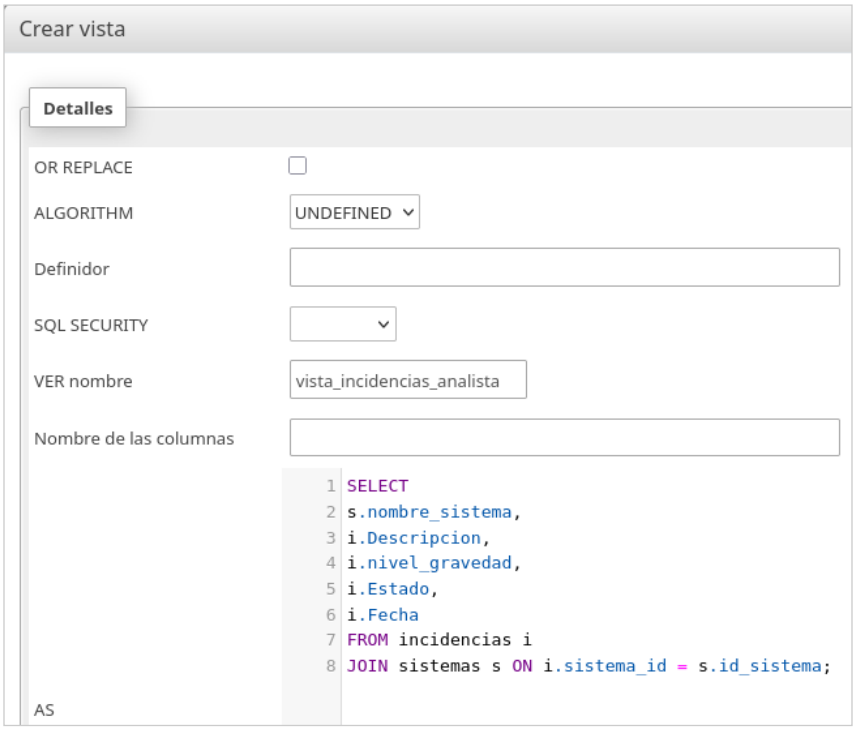
2.1. Creación vista_incidencias_analista

Esta vista permitirá al rol analista ver el estado general de las incidencias, sin mostrar la **IP del sistema**, **usuario responsable** y cualquier **dato personal**.

La podemos crear por comandos:

```sql
CREATE VIEW vista_incidencias_analista AS
SELECT
    s.nombre_sistema,
    i.Descripcion,
    i.nivel_gravedad,
    i.Estado,
    i.Fecha
FROM incidencias i
JOIN sistemas s ON i.sistema_id = s.id_sistema;
```

O de forma gráfica, desde cualquier tabla –> **Examinar** –> **Crear Vista**, donde introducimos la consulta SQL que normalmente haríamos para mostrar los datos que queremos ver:



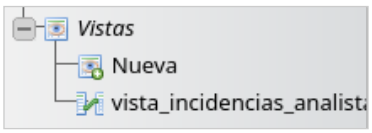
2.2. Creación vista_sistemas_sin_ip

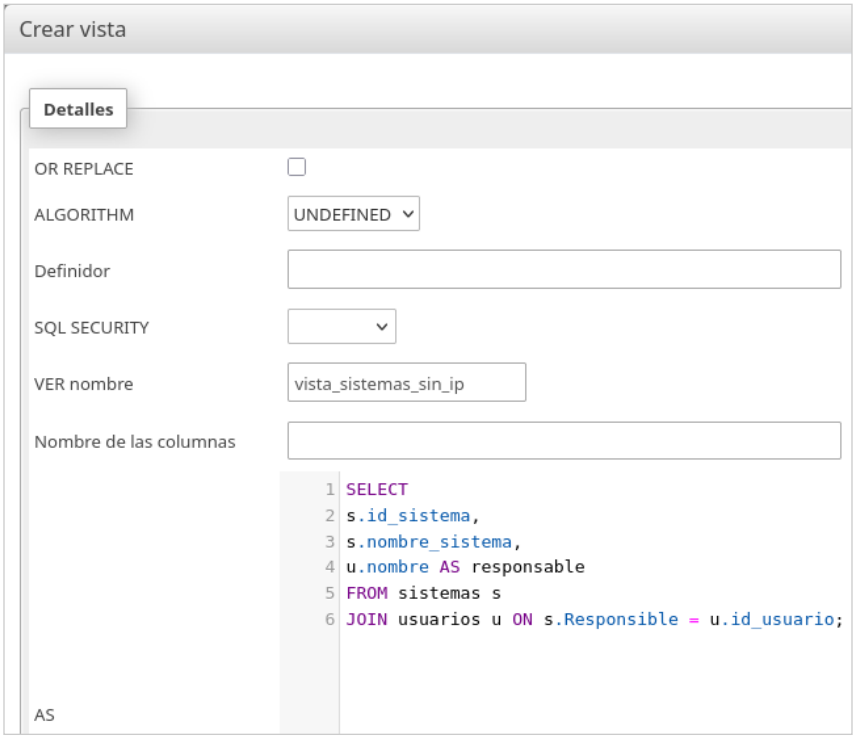
Esta vista permitirá a analistas y soporte conocer qué sistemas existen y quién los gestiona, pero sin mostrar **direcciones IP internas**.

La podemos crear por comandos:

```sql
CREATE VIEW vista_sistemas_sin_ip AS
SELECT
    s.id_sistema,
    s.nombre_sistema,
    u.nombre AS responsable
FROM sistemas s
JOIN usuarios u ON s.Responsible = u.id_usuario;
```

O de forma gráfica, como ya hay una vista creada habrá una nueva sección, accediendo desde **Vistas** –> **Nueva** donde introducimos la consulta SQL que normalmente haríamos para mostrar los datos que queremos ver:





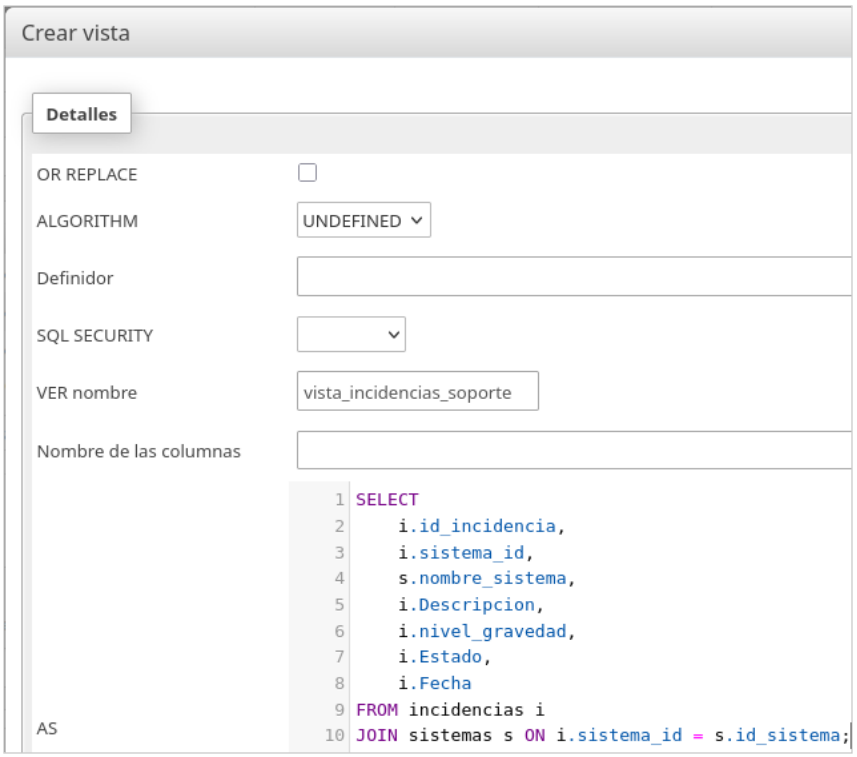
2.3. Creación vista_incidencias_soporte

Esta vista mostrará a los técnicos de soporte las incidencias en curso, solo de los sistemas que gestionan.

La podemos crear por comandos:

```sql
CREATE VIEW vista_incidencias_soporte AS
SELECT
    i.id_incidencia,
    i.sistema_id,
    s.nombre_sistema,
    i.Descripcion,
    i.nivel_gravedad,
    i.Estado,
    i.Fecha
FROM incidencias i
JOIN sistemas s ON i.sistema_id = s.id_sistema;
```

O de forma gráfica, como ya hay una vista creada, accederemos desde **Vistas** –> **Nueva** donde introducimos la consulta SQL que normalmente haríamos para mostrar los datos que queremos ver:



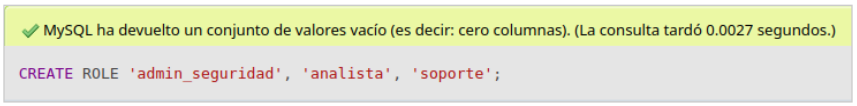
3. Creación de Usuarios y Modelo RBAC

3.1. Creación de Roles

En phpMyAdmin no existe un botón o menú visual específico que nos permita escribir o crear un objeto ROLE de SQL mediante clics de forma nativa. Por lo que se crearán los roles por comandos:

SQL

```sql
CREATE ROLE 'admin_seguridad', 'analista', 'soporte';
```



Concesión de Privilegios

SQL

```sql
GRANT ALL PRIVILEGES ON DataShield_DavidMarco.* TO 'admin_seguridad';

GRANT SELECT ON DataShield_DavidMarco.vista_incidencias_analista TO 'analista';
GRANT SELECT ON DataShield_DavidMarco.vista_sistemas_sin_ip TO 'analista';

GRANT SELECT, UPDATE ON DataShield_DavidMarco.vista_incidencias_soporte TO '
    soporte';
GRANT SELECT ON DataShield_DavidMarco.vista_sistemas_sin_ip TO 'soporte';
```



3.2. Creación de Usuarios

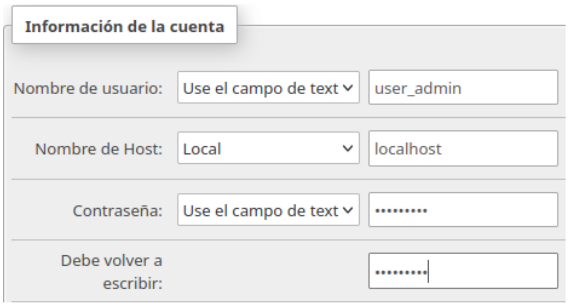
Los usuarios, con el fin de probar la seguridad que posee la base de datos, recibiran los siguientes nombres y roles:

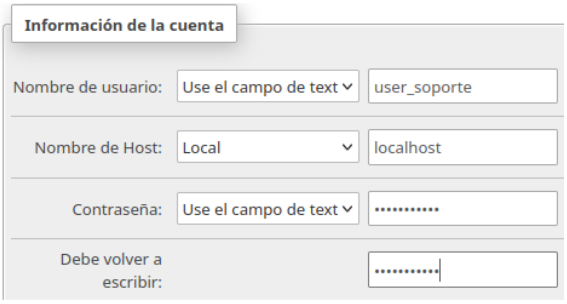


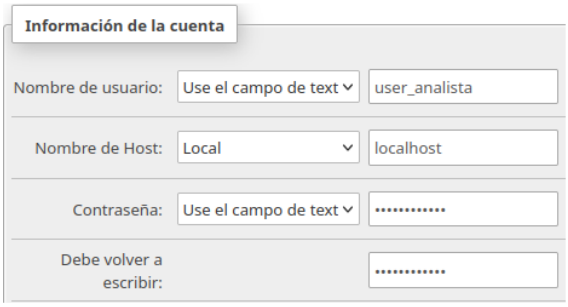
Podemos crearlos mediante comandos:

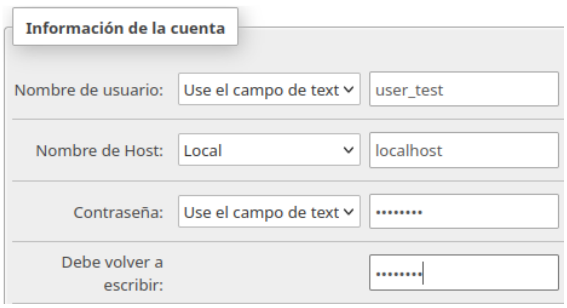
```sql
CREATE USER 'user_admin'@'localhost' IDENTIFIED BY 'Admin123*';
CREATE USER 'user_analista'@'localhost' IDENTIFIED BY 'Analista123*';
CREATE USER 'user_soporte'@'localhost' IDENTIFIED BY 'Soporte123*';
CREATE USER 'user_test'@'localhost' IDENTIFIED BY 'Test123*';
```

O de forma gráfica, accediendo desde **Servidor: localhost** –> **Cuentas de usuarios** –> **Agregar cuenta de usuario**.









Concesión de Roles

En phpMyAdmin no existe un botón o menú visual específico que nos permita asignar roles a usuarios de forma nativa. Por lo que se asignarán los roles por comandos:

```sql
GRANT 'admin_seguridad' TO 'user_admin'@'localhost';
SET DEFAULT ROLE 'admin_seguridad' FOR 'user_admin'@'localhost';

GRANT 'analista' TO 'user_analista'@'localhost';
SET DEFAULT ROLE 'analista' FOR 'user_analista'@'localhost';

GRANT 'soporte' TO 'user_soporte'@'localhost';
SET DEFAULT ROLE 'soporte' FOR 'user_soporte'@'localhost';

FLUSH PRIVILEGES;
```



> Con el comando **SET DEFAULT ROLE ALL TO** se quedarán guardados los roles para los usuarios, teniendolo asignado siempre que inicie sesión. Pero esto solo está en versiones posteriores a la 10.5.0 de MariaDB.

4. Verificación de la configuración asignada

Para poder cambiar de usuario en phpMyAdmin habrá que hacer un cambio en sus archivos, primero de todo accediendo a él:

TERMINAL

```
usuario@PC-usuario:~$ sudo nano /opt/lampp/phpmyadmin/config.inc.php
```

En este archivo que contiene la configuración de acceso, ubicaremos el apartado de **Authentication type** y la instrucción `$cfg['Servers'][$i]['auth_type'] = 'config';`, donde cambiaremos **config** por **cookie**.

```
$cfg['Servers'][$i]['auth_type'] = 'cookie';
```

De esta forma, al acceder a phpMyAdmin, no se accederá como root, sino que te pedirá los datos de usuario y contraseña para que inicies sesión con la cuenta que quieras.

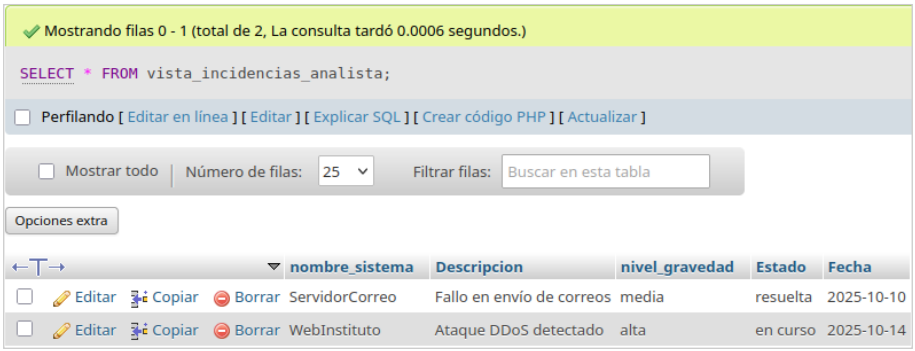


4.1. Pruebas funcionales básicas

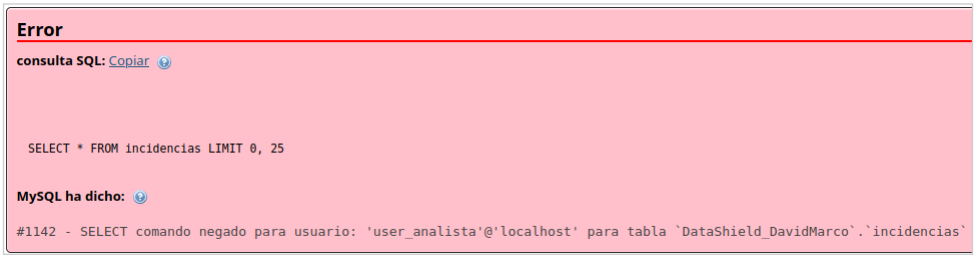
Acceso como user_analista

Iniciaremos como **user_analista** y haremos las siguientes consultas:

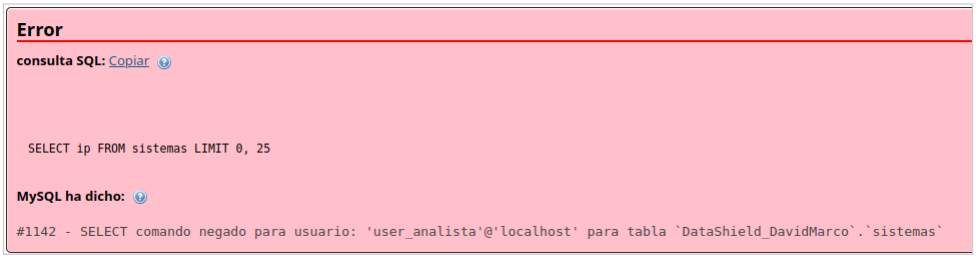
```sql
SELECT * FROM vista_incidencias_analista;
SELECT * FROM incidencias;
SELECT ip FROM sistemas;
```



Como poseemos el rol **analista** podremos ver la vista **vista_incidencias_analista**.



Como poseemos el rol **analista** no podremos ver la tabla **incidencias**.

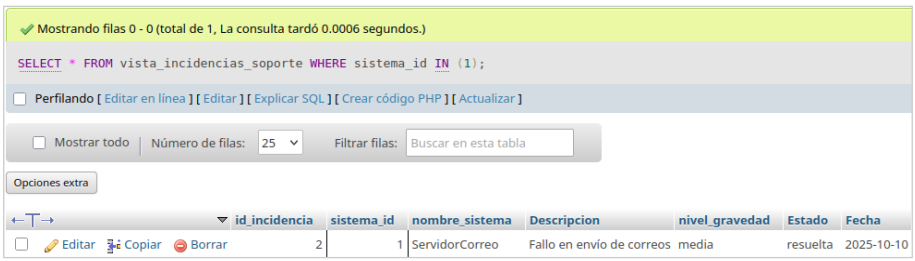


Como poseemos el rol **analista** no podremos ver la tabla **sistemas**.

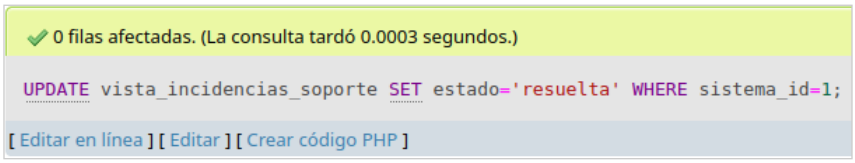
Acceso como user_soporte

Iniciaremos como **user_soporte** y haremos las siguientes consultas:

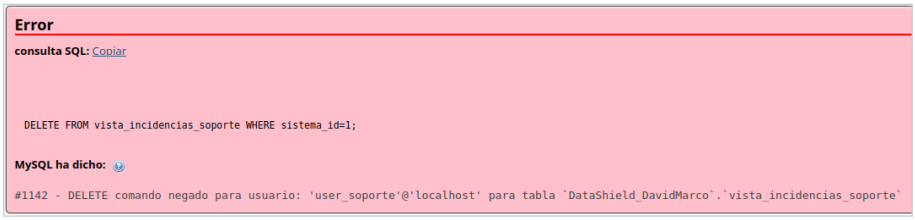
```sql
SELECT * FROM vista_incidencias_soporte WHERE sistema_id IN (1);
UPDATE vista_incidencias_soporte SET estado='resuelta' WHERE sistema_id=1;
DELETE FROM vista_incidencias_soporte WHERE id_sistema=2;
UPDATE sistemas SET ip='172.0.0.1' WHERE id_sistema=2;
```



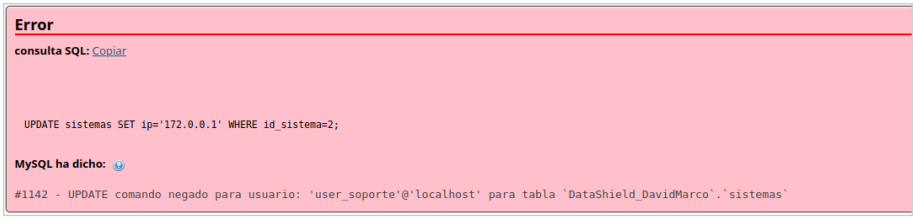
Como poseemos el rol **soporte** podremos ver la vista **vista_incidencias_soporte**.



Como poseemos el rol **soporte** podremos hacer **UPDATE** en la vista **vista_incidencias_soporte**.



Como poseemos el rol **soporte** no podremos hacer **DELETE** en la vista **vista_incidencias_soporte**.

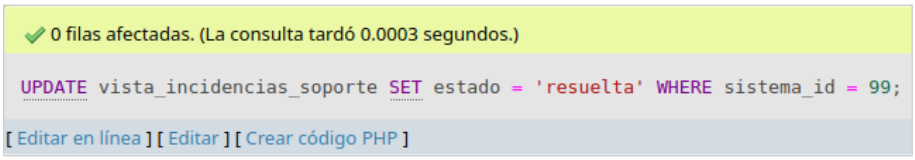


Como poseemos el rol **soporte** no podremos hacer **UPDATE** en la tabla **sistemas**.

4.2. Pruebas de separación de datos

Accederemos con el usuario **user_soporte** haremos las siguientes consultas:

```sql
UPDATE vista_incidencias_soporte  SET estado = 'resuelta'  WHERE sistema_id = 99
    ;
SELECT * FROM vista_incidencias_soporte;
```



No se ve afectado ya que el sistema no existe. Si existiese se vería afectado ya que poseemos el rol de **soporte**, el cual tiene permisos de edición.



Como poseemos el rol **soporte** podremos ver la vista **vista_incidencias_soporte**.